

Security boundaries and recovery

Website documentation for jakeardoin.com

Prepared for the Django, Wagtail, and AWS website setup.

Keep this PDF with your password manager and deployment notes. The Markdown file remains the editable source.

Security boundaries and recovery

What the implementation enforces

The public site is readable without signing in. All authenticated requests, including editor, protected page, and document routes, require a password record and a recent WebAuthn verification in the server-side session. A live authenticator belonging to the signed-in user must match that record. Password-only legacy sessions cannot reach Wagtail. Old admin login/reset routes point to the common login.

django-allauth and Yubico's python-fido2 library verify WebAuthn challenges, signatures, RP IDs, and presence. Site form extensions enforce the exact configured origin; expired/missing challenges are rejected and successful challenges cannot be reused. The enrollment browser request selects a cross-platform authenticator and rejects backup-eligible synced passkeys. Manufacturer attestation is not required, so enrollment is not cryptographic proof of the Yubico brand. The user must register actual YubiKeys.

Public signup, passwordless login, remembered-device MFA bypass, TOTP, and recovery-code fallback are disabled. Password reset does not remove authenticators. The last key cannot be removed through its HTTP endpoint, with database locking serializing concurrent removals. Deleting a key revokes sessions relying on that key on their next request. Password changes invalidate Django sessions. Password and MFA freshness last one hour; challenge freshness lasts five minutes. Browser closure is requested as session expiry, but browser session restoration can preserve cookies, so explicit logout is still appropriate.

First enrollment is trusted provisioning: a server-created user's initial password is sufficient to enroll its first key. Until registration and subsequent proof of possession finish, Wagtail remains blocked. Enroll immediately and protect the initial password. Additional enrollment requires an existing verified session. A compromised browser or administrator/server can still alter site state; MFA does not protect a stolen active session or an already-compromised server.

Production uses HTTPS, secure cookies, CSRF checking, and shared database-backed login rate limits across application workers. Only Caddy publishes ports; EC2 allows public 80/443, not SSH or the database. Human server access uses AWS Session Manager. The instance role supplies temporary S3 credentials; it can access only the specified media/backup buckets plus SSM agent services. IMDSv2 is required with a hop limit of two for container access. All trusted workloads on this single host share the role; an application compromise can therefore affect its files/backups.

AWS/root/GitHub/domain/email MFA settings are external account configuration. Repository code cannot prove they are enabled. Existing API tokens and recovery methods are independent access paths; review them. Machine requests use narrowly scoped roles/credentials, not a key touch for every request. Do not claim every account is YubiKey-only merely because a browser key was registered.

Application audit events cover login, logout, failed login, key add/remove/use, and emergency recovery, without credentials. Rejected expired challenges are logged. Wagtail maintains its own content history. AWS CloudTrail records management API calls. Session transcripts and durable log export are additional operational configuration; local container logs are size-limited and are not a permanent audit archive.

Automated and manual validation

Run `python manage.py test tests` in a development/test environment. Security tests use ephemeral signed software credentials to exercise protocol verification, origin/challenge rejection, expired/replayed requests, last-key removal, revocation, wrong-user binding, and password-only access denial. They do not emulate physical YubiKey enrollment or prove hardware attestation.

Before launch, perform the real password-cancel/key-success checks in the AWS guide with both physical keys. CloudFormation validation and container checks are not proof of a successful AWS deployment; confirm stack creation, DNS, certificates, role access, S3 images, email, and an off-server restore in your account.

Recovery with a working AWS key

Use `changepassword` for a forgotten password without removing security keys. Use a backup website key for normal replacement. Only if every website key is lost, use the `recover_editor USERNAME --confirm-reset` management command through the protected AWS terminal. It replaces the password, deletes that user's key registrations, invalidates all website sessions, and logs the action. Immediately enroll/test replacement keys. Anyone with privileged server access can perform recovery, making AWS account protection essential.

Download an off-server backup

In the MFA-protected AWS console, open **S3** → **backup bucket** → **database/**, choose a dump, and copy its object key. In the server terminal after `sudo -i` and `cd /opt/mywebsite`, replace the filename below with that exact dump filename:

```
mkdir -p backups
chmod 700 backups
bash deploy/compose.sh run --rm --no-deps -T --user 0:0 -v "$PWD/backups:/backups" web python -c 'import
os,sys,boto3; name=sys.argv[1]; boto3.client("s3").download_file(os.environ["BACKUP_BUCKET_NAME"],
"database/"+name,"/backups/"+name); os.chmod("/backups/"+name,0o600)' YOUR_BACKUP.dump
```

Run the isolated test-database restore from the AWS guide before using a dump on live data.

Restore the live database deliberately

This replaces live content with the chosen backup and loses changes made after that backup. Do not run it as a routine deployment. Ensure you have the matching code version, the same original `.env.production` values, the media bucket, and a separately saved current dump. Restoring the database can also restore old user/password/key state, so inspect users and keys afterward.

- 1 Enter the protected AWS terminal, become root, and `cd /opt/mywebsite`.
- 2 Stop background writes: `systemctl stop website-backup.timer`. If you configured publishing cron, use `crontab -e` to temporarily comment out the website jobs by putting `#` at each line's start.
- 3 Run `bash deploy/backup.sh` to preserve current data off-server before replacing it.
- 4 Stop the application: `bash deploy/compose.sh stop web`. Caddy will show an error page during the maintenance window.
- 5 Replace `YOUR_BACKUP.dump` with the tested backup filename and run:

```
bash deploy/compose.sh exec -T db pg_restore -U website -d website --clean --if-exists --no-owner --
single-transaction --exit-on-error < backups/YOUR_BACKUP.dump
```

- 1 Before restarting the web service, remove all restored sessions so old cookies from the backup cannot be reused:

```
bash deploy/compose.sh exec -T db psql -U website -d website -c 'DELETE FROM django_session;'
```

- 1 Check out the matching code revision if necessary with `git checkout COMMIT_SHA`, replacing `COMMIT_SHA` with the recorded full revision. Do not assume any old code works with any newer schema.
- 2 Run `bash deploy/release.sh`. This rebuilds that revision, recreates the cache table if needed, runs any required migrations, and restarts the app.

- 3 Verify content, image delivery, and both keys. Review user accounts and remove credentials that should no longer exist. Re-register replacement keys if the restored backup predates their enrollment.
- 4 Restart the backup timer and restore the cron lines: `systemctl start website-backup.timer`, then `crontab -e`. Make a new successful backup.

If the server was completely lost, create a replacement from the template using the existing media bucket, recreate the saved environment, and build/start the database before downloading/restoring. Transfer or update DNS to its address. Keep the original backup bucket name in the environment and authorize the new instance role to read that bucket (and write new backups if continuing to use it); the new template's role initially targets its newly created backup bucket. That IAM change is an explicit administrator step, not an automatic cross-bucket permission. Preserve retained disks and buckets until the replacement is verified.

For a code-only rollback with no database changes, check out a known-good revision and run the release script, then test. If migrations changed, use a compatible forward fix or a tested database restore. Django migrations are not automatically reversible, and restores discard newer edits.